Marshaling Objects

Ruby can take an object and convert it into a stream of bytes that can be stored outside the application. This process is called *marshaling*. This saved object can later be read by another instance of the application (or by a totally separate application), and a copy of the originally saved object can be reconstituted.

Two potential issues arise when you use marshaling. First, some objects can't be dumped. If the objects to be dumped include bindings, procedure or method objects, instances of the `IO` class, or singleton objects—or if you try to dump anonymous classes or modules—a `TypeError` will be raised.

Second, when you load a marshaled object, Ruby needs to know the definition of the class of that object (and of all the objects it contains).

Rails uses marshaling to store session data. If you rely on Rails to dynamically load classes, it's possible that a particular class may not have been defined at the point it reconstitutes session data. For that reason, use the `model` declaration in your controller to list all models that are marshaled. This preemptively loads the necessary classes to make marshaling work.

Now that you have the Ruby basics down, let's give what we learned a whirl with a slightly larger, annotated example that pulls together a number of concepts. We'll follow that with a walk-through of special features that will help you with your Rails coding.